

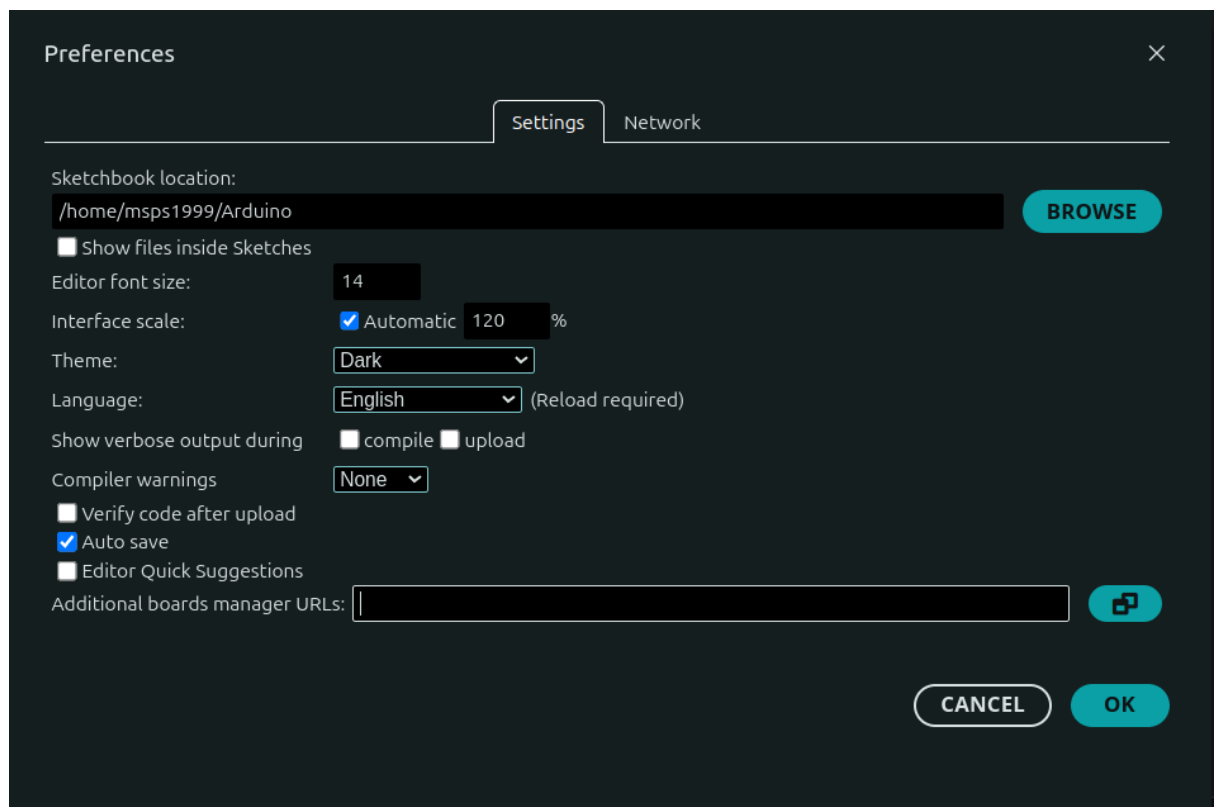
Разработка прошивки для ESP32-CAM с подключением к Wi-Fi

1. Проверка установленных библиотек в IDE

Данный раздел нужен для устранения распространенных проблем, которые могут возникнуть во время начала работы над проектом. Если у вас все хорошо, то переходите к пункту два.

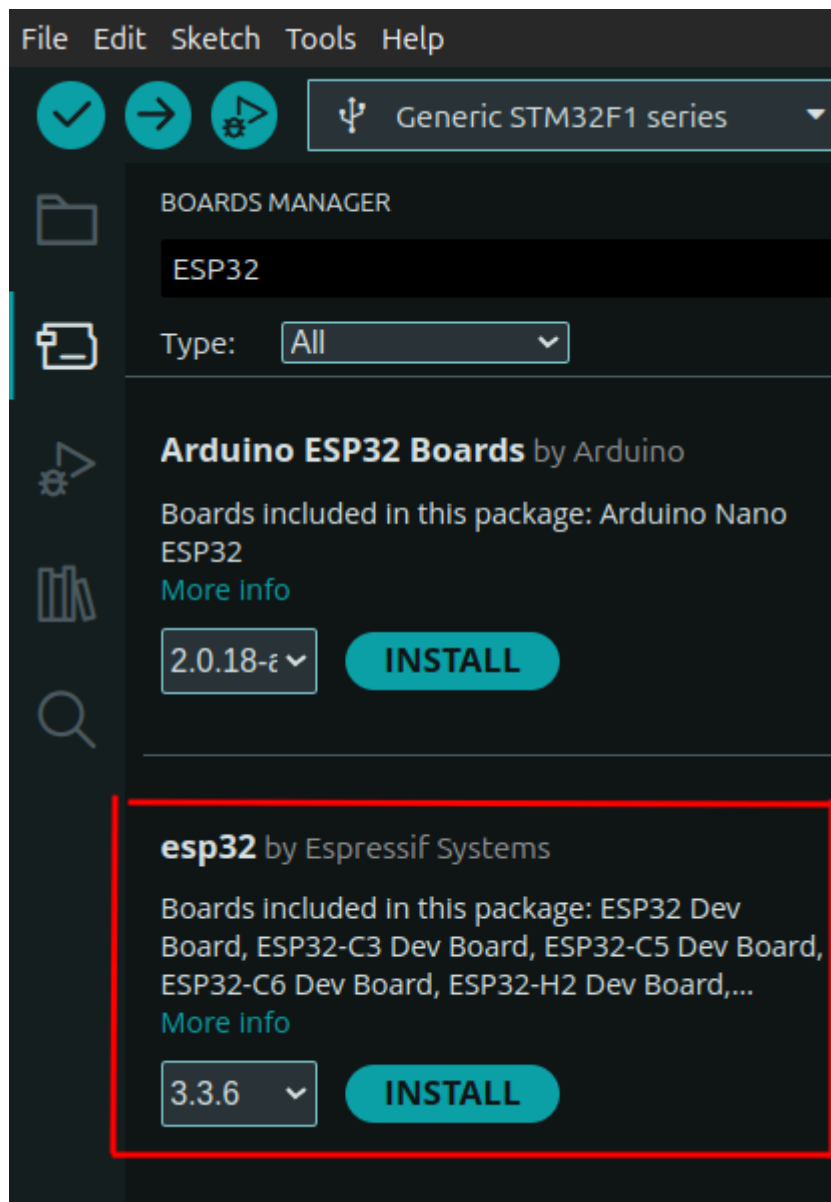
Разработку прошивок для ESP32 мы будем проводить в среде разработки Arduino IDE. Студенты могут использовать и другие варианты сред, но их мы далее рассматривать не будем, поэтому с сопутствующими проблемами придется разбираться либо самостоятельно, либо с помощью наставников.

Первым делом при запуске Arduino IDE нужно перейти в меню Файл->Настройки. У вас откроется следующее меню:



В поле “Дополнительные ссылки для менеджера плат” или же в английской версии “Additional boards manager URLs” нужно вставить следующую строку:

https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json, которая нам позволит скачать нужные модули для разработки под ESP32.

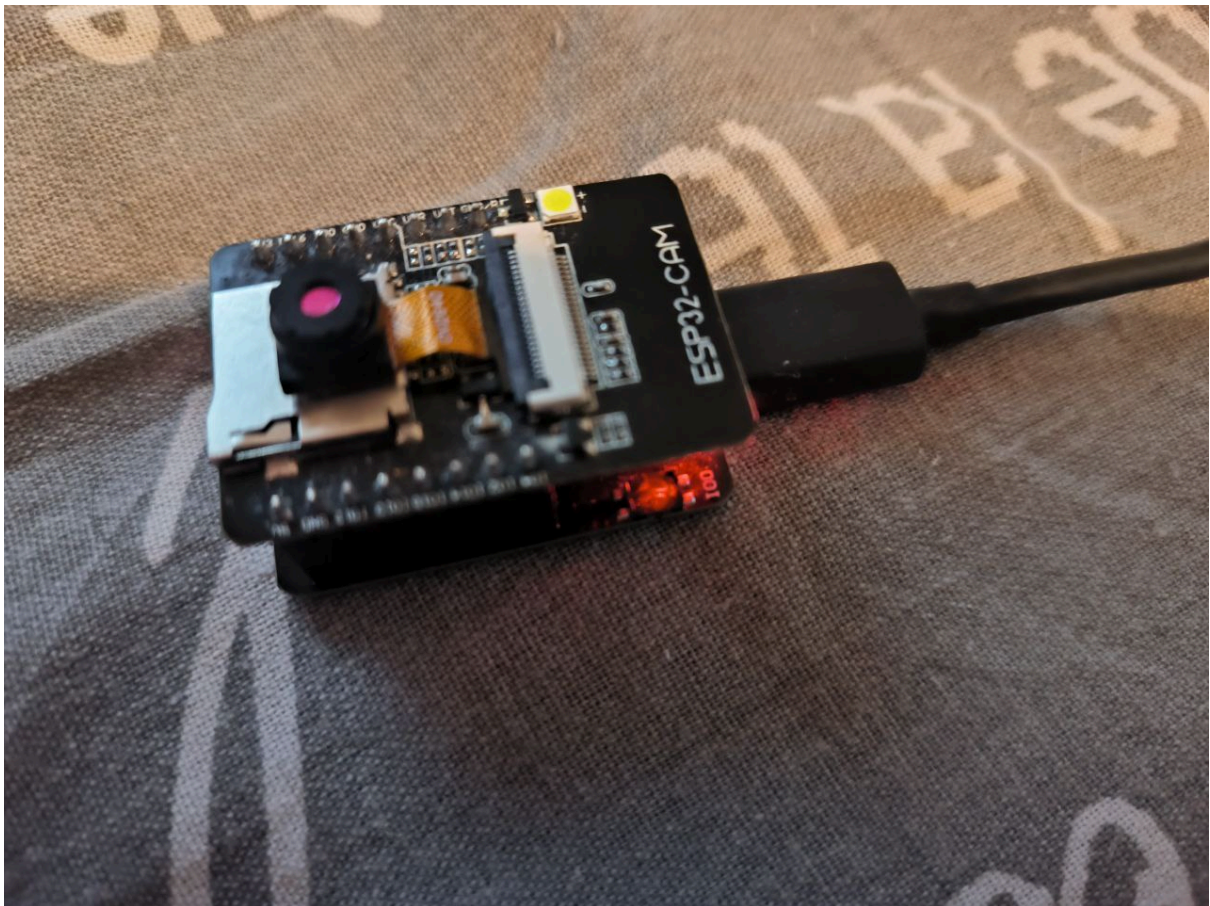


На боковой панели есть меню “Менеджер плат”, в котором мы можем скачать необходимые нам инструменты. В поиске вводим “ESP32” и скачиваем выделенный красным пакет. Если пакет у вас установлен, то вместо “INSTALL” будет написано “REMOVE”.

На данный момент мы проверили все основные моменты, связанные с настройкой IDE.

2. Настройка подключения к плате

Переходим в меню “Инструменты” -> “Плата” -> “esp32” и выбираем нужную нам модель платы. В нашем случае это “ESP32 Dev Module”. Теперь нам нужно указать, к какому порту мы подключены по USB. В меню “Инструменты” -> “Порт” выбираем COM-порт, к которому подключена плата (например COM1 в Windows или /dev/ttyUSB0 в Linux)



Создайте пустой скетч (“Файл”->”Новый скетч”) и попробуйте загрузить программу через кнопку “Загрузить” в верхнем левом углу. Если вы увидите в консоли, что показано на скриншоте ниже, то вы все сделали правильно. Если возникли проблемы, проверьте правильность порта, либо вытащите USB и вставьте его обратно.

```
Output
Sketch uses 269299 bytes (20%) of program storage space. Maximum is 1310720 bytes.
Global variables use 20792 bytes (6%) of dynamic memory, leaving 306888 bytes for local variables. Maximum is 32768 bytes.
esptool v5.1.0
Serial port /dev/ttyUSB0:
Connecting.....
Connected to ESP32 on /dev/ttyUSB0:
Chip type:      ESP32-D0WD-V3 (revision v3.1)
Features:       Wi-Fi, BT, Dual Core + LP Core, 240MHz, Vref calibration in eFuse, Coding Scheme None
Crystal frequency: 40MHz
MAC:            fc:b4:67:4e:8e:04

Uploading stub flasher...
Running stub flasher...
Stub flasher running.
Changing baud rate to 921600...
Changed.

Configuring flash size...
Flash will be erased from 0x00001000 to 0x00007fff...
Flash will be erased from 0x00008000 to 0x00008fff...
Flash will be erased from 0x00009000 to 0x00009fff...
Flash will be erased from 0x0000a000 to 0x0000afff...
Flash will be erased from 0x0000b000 to 0x0000bfff...
```

3. Разработка прошивки

Наконец мы приступили к разработке прошивки!

Продолжаем работу в скетче, на котором мы проверяли подключения платы к компьютеру. Первым делом подключаем необходимые библиотеки для работы с сетью:

```
#include <WiFi.h>
#include <HTTPClient.h>
```

Если кратко, то библиотеки нужны для следующего:

- **#include <WiFi.h>** - библиотека для работы с Wi-Fi
- **#include <HTTPClient.h>** - библиотека для HTTP-запросов

На плате есть светодиод, который мы можем использовать как индикатор, во-первых, работы платы, во-вторых, успешной отправки данных на локальный сервер. Займемся его настройкой.

Первым делом сделаем глобальную переменную, указывающую на номер пина, который управляет светодиодом:

```
#include <WiFi.h>
#include <HTTPClient.h>
```

```
#define LED_PIN 4 // Встроенный светодиод на ESP32-CAM
```

```
void setup() {  
}
```

Теперь перейдем к функции “*setup()*”. Данная функция вызывается в самом начале работы прошивки и срабатывает ровно один раз. Ее используют для настройки различных аспектов программы, например, настройки портов ввода-вывода, инициализации драйверов и т.д. Мы в функции “*setup()*” настроим пин управления светодиодом и последовательный порт для отладки работы прошивки. С помощью последовательного порта будем отправлять логи:

```
// Инициализация последовательного порта для отладки  
Serial.begin(115200);  
Serial.println("Инициализация");  
// Настройка светодиода  
pinMode(LED_PIN, OUTPUT);  
digitalWrite(LED_PIN, LOW);
```

По коду можно увидеть, что пин настраиваем (функция *pinMode*) на “Выход” - т.е. пин может управлять напряжением. Напряжение установим в “LOW” (функция *digitalWrite*), т.е. низкое напряжение на выходе ножки.

Задание: Чтобы светодиод загорелся, попробуйте выставить “высокое” напряжение на ножке с помощью “HIGH”

Замечание: при долгом горении светодиода плата может сильно нагреваться - следите за этим! При сильном нагреве отключите светодиод в коде и пользуйтесь только отладкой!

Замечание: Чтобы видеть вывод от *Serial.println("Инициализация")* включите в IDE Serial Monitor, нажав комбинацию клавиш Ctrl+Shift+M.

Перейдем к подключению к Wi-Fi сети. Создадим функцию *connectToWiFi()* для удобства.

Внутри нее вызовем метод *begin()* объекта *WiFi*, который по указанному имени сети и паролю, позволит подключиться к ней:

```
void connectToWiFi() {  
    // Включаем Wi-Fi  
    WiFi.begin("ИМЯ_СЕТИ", "ПАРОЛЬ_СЕТИ");  
}
```

Для подключения к сети может потребоваться некоторое время, поэтому мы напишем цикл, который будет проверять статус подключения, и отображать это в виде мигания светодиодом:

```
// Ждем подключения (максимум 20 секунд)  
int attempts = 0;  
while (WiFi.status() != WL_CONNECTED && attempts < 40) {  
    delay(500);  
    Serial.print(".");  
    digitalWrite(LED_PIN, !digitalRead(LED_PIN)); // Мигаем светодиодом  
    attempts++;  
}
```

Рассмотрим данный код:

- 1) С помощью *WiFi.status() != WL_CONNECTED* мы проверяем подключились ли мы к сети или нет. Если нет, то ждем дальше.
- 2) *attempts < 40* - это условие, при котором попытки подключения будут прекращены, т.е. 40 попыток - это максимум. Вы можете вписать свое значение
- 3) *delay(500)* - мы ждем 500 мс, прежде чем идти дальше по коду;
- 4) *digitalWrite(LED_PIN, !digitalRead(LED_PIN))* - необычная на первый взгляд конструкция. Мы используем уже знакомую нам функцию *digitalWrite*, которая позволяет изменять уровень сигнала на пине.

Вопрос: почему мы вместо “LOW”/”HIGH” используем функцию *digitalRead(LED_PIN)*, если учесть, что в языке C++ “!” это оператор “Не”?

Ответ:

Каждый раз мы считываем значение, которое установлено на пине, и

устанавливаем его в противоположное, т.к. уровень сигнала - это булевская переменная, которая может иметь либо значение, равное "1", либо "0". Если не понятно, подойдите к наставнику.

Распространенные проблемы при подключении к Wi-Fi:

- 1) Проверьте SSID и пароль
- 2) Убедитесь, что сеть работает на частоте 2.4 ГГц

Независимо от результата подключения, мы должны уведомить об этом пользователя. Выведем результат в консоль, а также зажжем/погасим светодиод:

```
// Проверяем результат
if (WiFi.status() == WL_CONNECTED) {
  Serial.print("IP адрес: ");
  Serial.println(WiFi.localIP());
  digitalWrite(LED_PIN, HIGH); // Включаем светодиод
} else {
  digitalWrite(LED_PIN, LOW); // Выключаем светодиод
}
```

WiFi.localIP() позволяет нам узнать IP адрес сервера, к которому мы подключились.

Задание: подумайте, какие логи (сообщения в консоль) вам нужны, чтобы вам было удобнее отлаживать код? Добавьте себе в код данные выводы.

Ответ: Например:

- 1) Вывод о начале подключения к WiFi;
- 2) Имя сети, к которой идет подключение;
- 3) Пароль сети;
- 4) Статус подключения к сети.

Функция готова! Добавьте ее в функцию *setup()*:

```
void setup() {
```

```
// Инициализация последовательного порта для отладки  
Serial.begin(115200);  
Serial.println("Инициализация");  
// Настройка светодиода  
pinMode(LED_PIN, OUTPUT);  
digitalWrite(LED_PIN, LOW);  
// Подключаемся к Wi-Fi  
connectToWiFi();  
}
```

После запуска программы в консоли должно быть что-то вроде этого:

```
20:54:19.286 -> Инициализация  
20:54:19.286 ->  
20:54:19.863 -> ....  
20:54:21.371 -> IP адрес: 192.168.31.138
```

На данный момент все. В следующем уроке мы попробуем подключиться к серверу и передать данные на него.